

Modul 12 Praktikum Natural Language Processing

Ujian Akhir Semester Pemrosesan Bahasa Alami (Natural Language Processing)

Deskripsi Singkat

Pada UAS ini, praktikan akan membangun sebuah aplikasi berbasis web yang memungkinkan pengguna berinteraksi melalui suara dan menerima respons suara kembali dari sistem. Aplikasi ini menggabungkan teknologi Speech-to-Text (STT), Large Language Model (LLM), dan Text-to-Speech (TTS) untuk membentuk chatbot berbasis suara yang responsif dan komunikatif.

Praktikan akan melengkapi kode-kode backend menggunakan **FastAPI** untuk memproses input audio, mentranskripsinya menjadi teks menggunakan model **Whisper**, menghasilkan respons menggunakan **Google Gemini API**, dan mengubah hasil teks menjadi suara kembali menggunakan model **Coqui TTS**. Antarmuka pengguna dikembangkan menggunakan **Gradio**, yang memungkinkan pengguna merekam suara langsung dari mikrofon dan mendengarkan balasan suara dari sistem. Proyek ini melatih praktikan dalam membangun pipeline NLP end-to-end dengan integrasi multimodal.

Tujuan

1. Membangun aplikasi chatbot berbasis web yang memungkinkan interaksi dua arah menggunakan suara.
2. Mengintegrasikan proses Speech-to-Text (STT), Large Language Model (LLM), dan Text-to-Speech (TTS) dalam satu pipeline percakapan.
3. Menghubungkan backend dengan model **Whisper** untuk transkripsi suara dan **Coqui TTS** untuk sintesis suara.
4. Menggunakan Google Gemini API untuk menghasilkan respons otomatis yang cerdas dan relevan dalam Bahasa Indonesia.
5. Mengembangkan antarmuka pengguna menggunakan **Gradio** agar pengguna dapat merekam dan menerima audio secara langsung.
6. Melatih praktikan memahami alur data multimodal dan integrasi API dalam sistem NLP berbasis web.

Tools dan Teknologi yang Digunakan

- **FastAPI** — Framework backend berbasis Python yang digunakan untuk membuat REST API secara cepat dan efisien. Digunakan dalam proyek ini untuk menangani pipeline **backend** mulai dari *Speech-To-Text*, *Large Language Model*, hingga *Text-To-Speech*.
- **Gradio** — Library Python yang memungkinkan pembuatan UI berbasis web secara cepat. Digunakan untuk memungkinkan pengguna merekam suara secara langsung dan mendengarkan respons dari chatbot.
- **Whisper** — Model open-source dari OpenAI untuk Speech-to-Text (STT). Digunakan untuk mentranskripsi input suara dari pengguna menjadi teks. Dalam proyek ini digunakan versi binari (C++ CLI) agar ringan dijalankan di CPU.
- **Gemini API** — Layanan Large Language Model dari Google. Digunakan untuk menghasilkan respons berdasarkan transkripsi suara pengguna. API ini diakses melalui Python SDK resmi dari Google.
- **Coqui TTS** — Toolkit open-source untuk Text-to-Speech. Digunakan untuk mengubah output teks dari LLM menjadi audio dengan suara Bahasa Indonesia yang natural, menggunakan model dari repositori *indonesian-tts*.

Rangkaian Pengembangan dan Integrasi Sistem

Struktur Folder Proyek

Proyek ini terdiri dari beberapa direktori dan file utama sebagai berikut:

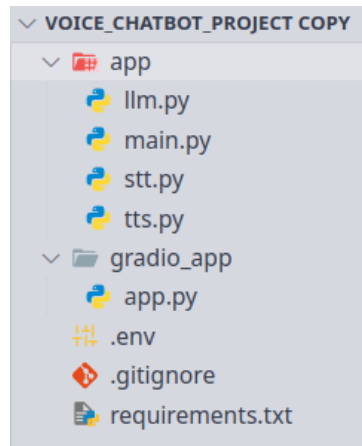


Figure 1: Struktur proyek

Folder `app/` berisi semua komponen backend seperti pemrosesan STT, TTS, dan LLM. Folder `gradio_app/` berisi antarmuka pengguna berbasis Gradio.

Pembuatan Virtual Environment dan Instalasi Dependensi

Langkah pertama sebelum menjalankan proyek adalah membuat virtual environment di direktori root proyek, yaitu `voice_chatbot_project/`. Setelah environment dibuat dan diaktifkan, unduh seluruh dependensi dengan menggunakan perintah `pip install -r requirements.txt`.

```
$ python3 -m venv env
$ source env/bin/activate (linux)
$ env/Scripts/activate (windows)
(env) $ pip install -r requirements.txt
```

Implementasi Speech-to-Text (STT)

Pada bagian ini, praktikan akan menginstal dan mengintegrasikan `whisper.cpp` sebagai sistem transkripsi suara ke teks (Speech-to-Text) menggunakan model Whisper dari OpenAI. Model Whisper dikenal memiliki kemampuan multibahasa dengan akurasi yang tinggi, termasuk dalam memahami Bahasa Indonesia.

Langkah pertama adalah mengkloning repositori `whisper.cpp` dari GitHub menggunakan perintah berikut:

```
$ git clone https://github.com/ggerganov/whisper.cpp.git
```

Setelah proses kloning selesai, periksa kembali lokasi folder `whisper.cpp`. Jika folder tersebut masih berada di luar direktori `app/`, pindahkan folder `whisper.cpp` ke dalam direktori `app/`. Pemindahan ini bertujuan agar seluruh proses yang berkaitan dengan backend—termasuk transkripsi audio—terorganisasi dan terpusat di dalam folder `app/`, sehingga memudahkan pengelolaan dan integrasi antar komponen aplikasi.

Langkah selanjutnya, masuk ke dalam folder `whisper.cpp` untuk mengunduh model Whisper yang akan digunakan. Untuk keperluan proyek ini, disarankan menggunakan model `ggml-large-v3-turbo` karena kemampuannya sudah bagus dalam memahami percakapan dalam Bahasa Indonesia. Namun, model ini cukup berat dijalankan pada CPU kelas menengah ke bawah. Oleh karena itu, praktikan dapat memilih model Whisper lain yang lebih ringan jika diperlukan.

Daftar lengkap model yang tersedia beserta ukuran dan tingkat akurasinya dapat dilihat pada gambar berikut:

Model	Disk	Mem
tiny	75 MiB	~273 MB
base	142 MiB	~388 MB
small	466 MiB	~852 MB
medium	1.5 GiB	~2.1 GB
large	2.9 GiB	~3.9 GB

Figure 2: Size dari model serta penggunaan memorynya

Setelah berhasil mengkloning repositori `whisper.cpp`, langkah selanjutnya adalah mengunduh model Whisper yang akan digunakan untuk proses transkripsi. Model ini diperlukan agar sistem dapat mengenali dan mengubah suara menjadi teks.

Model-model Whisper tersedia dalam berbagai ukuran seperti `tiny`, `base`, `small`, `medium`, dan `large`. Masing-masing memiliki trade-off antara ukuran model, akurasi, dan kebutuhan komputasi. Untuk keperluan tugas ini, praktikan disarankan menggunakan model `ggml-large-v3-turbo.bin` karena model ini memiliki performa yang baik dalam memahami Bahasa Indonesia, walaupun memerlukan sumber daya CPU yang lebih besar.

Untuk mengunduh model, jalankan perintah berikut dari dalam folder `whisper.cpp`:

```
$ cd whisper.cpp
$ ./models/download-ggml-model.sh large-v3-turbo
```

Jika menggunakan Windows, perintah serupa bisa dijalankan menggunakan Git Bash atau terminal dengan dukungan shell script. Setelah selesai, file model seperti `ggml-large-v3-turbo.bin` akan tersimpan di dalam folder `models/`.

Berikut adalah ilustrasi beberapa pilihan model yang tersedia dan dapat diunduh:

```
# Whisper models
models="tiny
tiny.en
tiny-q5_1
tiny.en-q5_1
tiny-q8_0
base
base.en
base-q5_1
base.en-q5_1
base-q8_0
small
small.en
small.en-tdrz
small-q5_1
small.en-q5_1
small-q8_0
medium
medium.en
medium-q5_0
medium.en-q5_0
medium-q8_0
large-v1
large-v2
large-v2-q5_0
large-v2-q8_0
large-v3
large-v3-q5_0
large-v3-turbo
large-v3-turbo-q5_0
large-v3-turbo-q8_0"
```

Figure 3: Size dari model serta penggunaan memorynya

Praktikan juga diperbolehkan menggunakan model yang lebih kecil seperti **base** atau **tiny** apabila perangkat yang digunakan memiliki keterbatasan sumber daya.

Setelah mengunduh model, langkah selanjutnya adalah melakukan proses build terhadap **whisper.cpp** agar dapat dijalankan di sistem Anda. Proses ini akan menghasilkan file eksekusi **whisper-cli** yang nantinya digunakan untuk melakukan transkripsi.

Gunakan perintah berikut sesuai dengan sistem operasi Anda:

Untuk Linux / macOS:

```
$ cmake -B build
$ cmake --build build --config Release
```

Untuk Windows:

```
cmake -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build --config Release
```

Setelah proses build selesai, lakukan uji coba awal menggunakan file audio contoh yang telah tersedia di dalam folder **samples/**. Berikut contoh perintah untuk menjalankan transkripsi:

```
$ ./build/bin/whisper-cli -m models/ggml-large-v3-turbo.bin -f samples/jfk.wav
```

Jika berhasil, sistem akan menampilkan transkrip dari file audio tersebut di terminal.

Langkah ini bertujuan untuk memastikan bahwa proses instalasi dan konfigurasi **whisper.cpp** sudah berjalan dengan benar sebelum diintegrasikan ke dalam aplikasi.

Setelah proses instalasi dan pengunduhan model selesai, langkah selanjutnya adalah mengintegrasikan **whisper.cpp** ke dalam sistem backend. Untuk itu, praktikan dapat membuka dan mulai mengonfigurasi file **stt.py**.

Pada file ini, terdapat sebuah fungsi utama yang bernama **transcribe_speech_to_text()** yang bertugas menjalankan **whisper-cli** secara langsung melalui kode Python menggunakan modul **subprocess**. Fungsi ini akan menerima input audio dari frontend, menyimpannya sementara sebagai file, lalu menjalankan proses transkripsi dan mengembalikan hasilnya dalam bentuk teks.

Setelah seluruh dependensi **whisper.cpp** tersedia, langkah selanjutnya adalah mengonfigurasi file **stt.py** agar dapat menjalankan fungsi transkripsi. Di dalam file ini, terdapat beberapa bagian kode yang perlu dilengkapi oleh praktikan, khususnya terkait jalur file menuju model dan binary **whisper-cli**.

Petunjuk pengisian sudah disediakan secara langsung di dalam file melalui komentar **TODO**, sehingga praktikan hanya perlu membaca, memahami, dan melengkapi bagian yang kosong sesuai dengan struktur direktori proyek masing-masing.

Latihan ini bertujuan agar praktikan terbiasa dalam mengatur struktur file proyek dan membangun path yang dinamis menggunakan modul **os**, sehingga kode tetap portabel dan tidak bergantung pada konfigurasi path yang statis.

Implementasi LLM (Gemini API)

Pada tahap ini, praktikan akan mengintegrasikan model Large Language Model (LLM) dari Google, yaitu Gemini, ke dalam backend aplikasi. Gemini akan berperan sebagai agen percakapan yang merespons hasil transkripsi suara dari pengguna. Integrasi dilakukan melalui API resmi dari Google Generative AI menggunakan Python SDK yang disediakan.

Untuk dapat menggunakan layanan Gemini API dalam proyek ini, praktikan perlu membuat API Key pribadi. Berikut langkah-langkahnya:

1. Buka browser dan telusuri **create gemini api key**. Klik tautan teratas dari situs resmi <https://aistudio.google.com>.

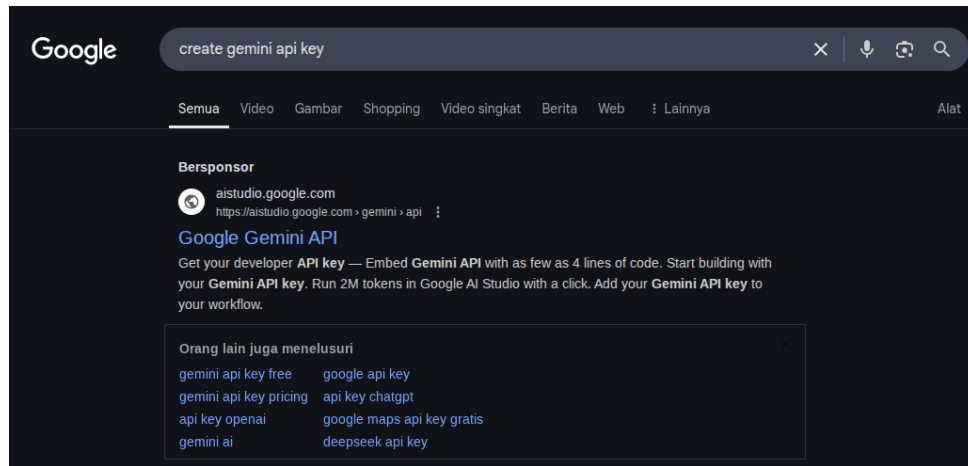


Figure 4: Menelusuri Gemini API Key di Google

2. Jika muncul pop up seperti dibawah ini, pilih tombol **Get API key**.

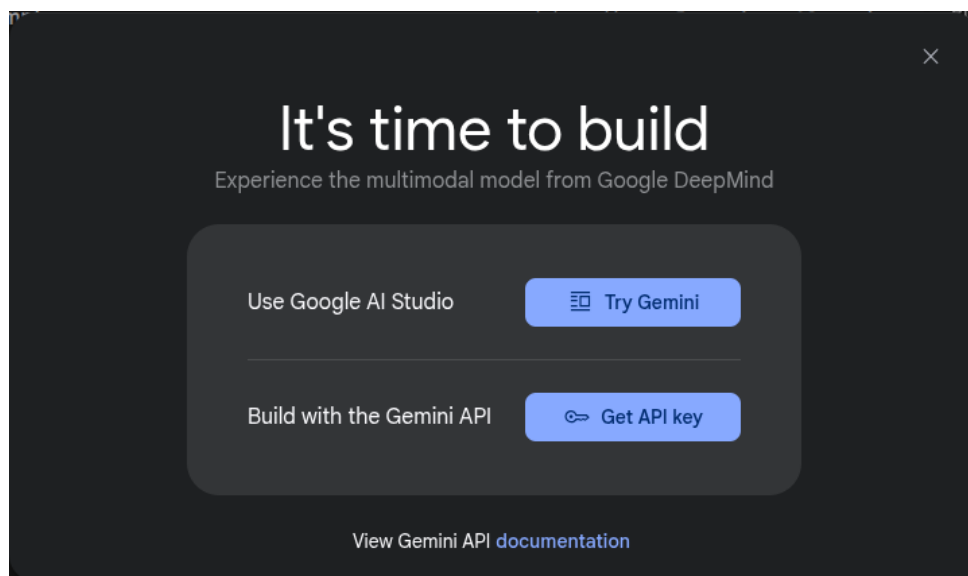


Figure 5: Tampilan awal pilihan membuat API Key

3. Praktikan akan diminta menyetujui *Terms of Service*. Centang dua kotak lalu klik tombol **I accept**.

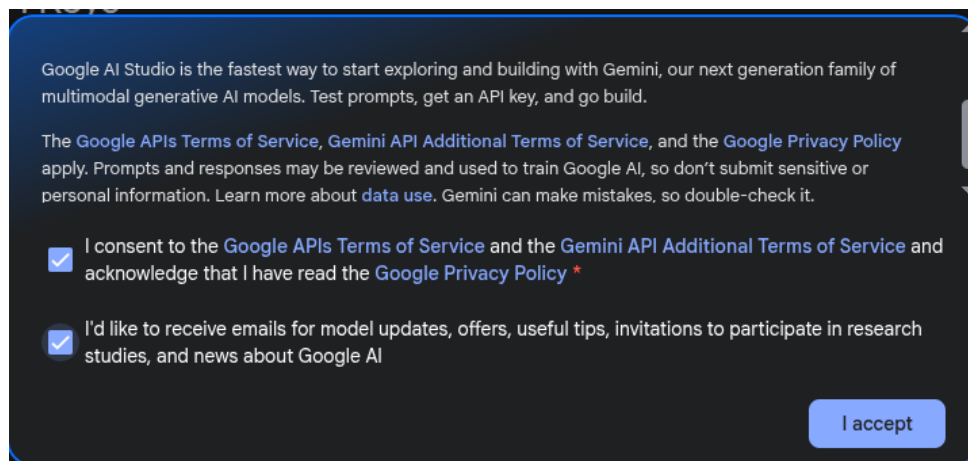


Figure 6: Persetujuan Terms of Service

4. Setelah itu, klik **Create API key** seperti tampak pada gambar berikut.

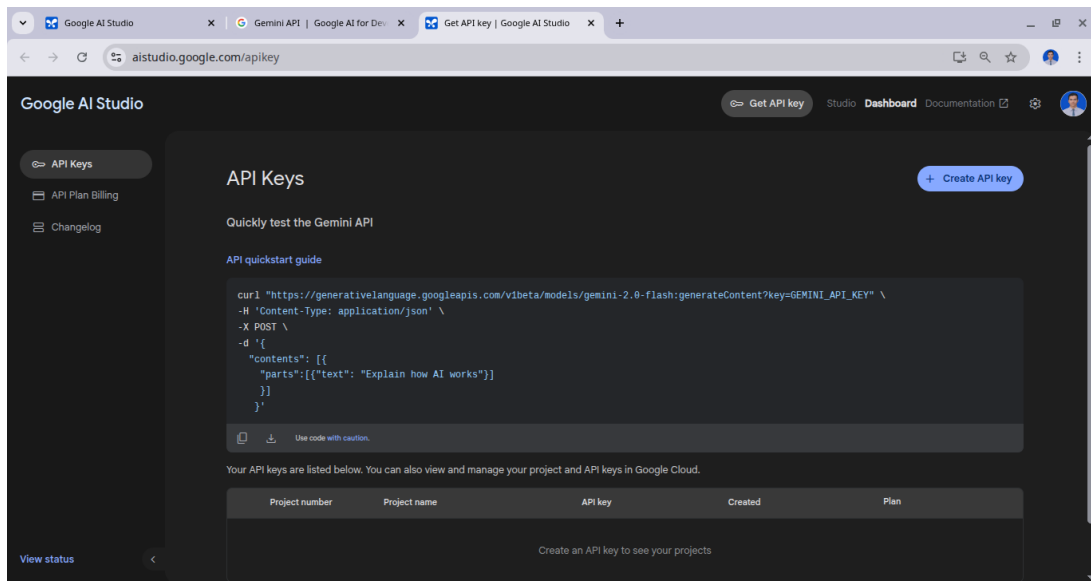


Figure 7: pertemuan12/Tampilan halaman API Key

5. Salin API key dan simpan dalam file `.env` didalam folder proyek.

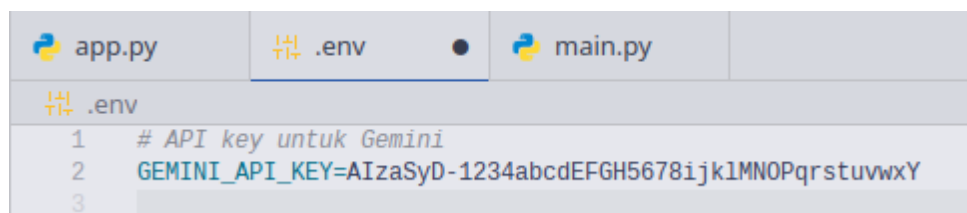


Figure 8: Contoh penulisan API Key di file `.env`

Hal ini untuk memastikan bahwa API key Anda valid dan tidak dibagikan secara publik.

Setelah API key berhasil disimpan, langkah selanjutnya adalah mengonfigurasi dan melengkapi kode program pada file `llm.py`. File ini bertanggung jawab untuk mengatur koneksi antara aplikasi dan model bahasa Gemini melalui API resmi dari Google.

Untuk membantu memahami struktur dan cara penggunaan **Google Generative AI Python SDK**, Anda disarankan untuk membaca dokumentasi resmi yang telah disediakan oleh Google melalui repositori **cookbook** berikut:

https://github.com/google-gemini/cookbook/blob/main/quickstarts/Get_started.ipynb

Dokumentasi tersebut menyajikan contoh kode lengkap mulai dari inialisasi client, konfigurasi **system instruction**, hingga pengiriman prompt ke model Gemini dan menerima responsnya. Anda dapat menyesuaikan struktur tersebut ke dalam file `llm.py` untuk kebutuhan proyek ini.

Implementasi TTS (Text-to-Speech)

Pada bagian ini, kita akan mengintegrasikan sistem Text-to-Speech (TTS) ke dalam aplikasi untuk mengubah respons teks dari model Gemini menjadi suara yang dapat didengar oleh pengguna. Model TTS yang digunakan dalam proyek ini adalah model **Indonesian-TTS** dari Wikidepia yang telah dilatih secara khusus untuk Bahasa Indonesia. Model ini bersifat open-source dan menggunakan arsitektur **VITS** yang mampu menghasilkan suara alami dan responsif terhadap fonem Bahasa Indonesia.

Proses implementasi akan dilakukan dengan mengunduh model, menguji fungsionalitas dasar menggunakan perintah terminal, serta menulis fungsi Python yang dapat menjalankan proses sintesis suara secara otomatis dari backend aplikasi.

Untuk menggunakan model **Indonesian-TTS**, silakan buka repositori GitHub resminya pada tautan berikut:

<https://github.com/Wikidemia/indonesian-tts>

Setelah masuk ke repository, navigasikan ke bagian yang menampilkan daftar versi model yang tersedia. Di sana, Anda akan melihat beberapa versi model yang dapat digunakan. Pilih versi **v1.2**. Setelah memilih versi tersebut, unduh file berikut:

- **checkpoint_1260000-inference.pth** – file model utama yang telah dilatih.
- **config.json** – file konfigurasi model yang menyimpan parameter dan pengaturan pelatihan.
- **speakers.pth** – file daftar speaker yang tersedia untuk digunakan dalam sintesis suara.

Setelah selesai mengunduh ketiga file tersebut, langkah selanjutnya adalah membuat folder baru bernama **coqui_utils/** di dalam folder **app/** pada struktur proyek Anda. Folder ini akan digunakan untuk menyimpan semua file yang berkaitan dengan model TTS. Pastikan Anda memindahkan ketiga file berikut ke dalam folder **coqui_utils/** yang baru saja dibuat. Folder **coqui_utils/** ini nantinya akan diakses langsung oleh backend untuk melakukan proses sintesis suara menggunakan model TTS yang telah Anda siapkan.

Sebelum mengintegrasikan model TTS ke dalam aplikasi backend, terlebih dahulu praktikan perlu memastikan bahwa model dapat dijalankan dengan baik secara manual melalui terminal.

Masuk ke direktori **coqui_tts/** melalui terminal dengan perintah berikut:

```
$ cd app/coqui_tts/
```

Selanjutnya, coba jalankan perintah yang terdapat di dalam githubnya untuk melakukan sintesis suara dari teks ke dalam file audio. Pastikan nama file **model** dan **config** telah sesuai dengan file yang telah Anda unduh sebelumnya.

Jika berhasil, akan dihasilkan sebuah file **output.wav** yang berisi hasil sintesis suara dari teks yang diberikan. Praktikan dapat memutar file ini untuk mengecek apakah model telah berjalan dengan baik. Terkait dengan kenapa teks yang digunakan memiliki bentuk yang aneh dikarenakan teks tersebut digunakan dalam format fonemik. Hal ini dikarenakan model TTS ini mengharapkan input berupa fonem untuk menghasilkan pelafalan yang lebih natural. Oleh karena itu, output dari LLM nantinya juga akan dikonversi ke bentuk fonem sebelum diproses oleh sistem TTS.

Buka file **tts.py** yang berada di dalam folder **app/**. Terdapat beberapa bagian kode yang telah dikosongkan dan perlu dilengkapi agar sistem dapat menjalankan proses Text-to-Speech dengan benar.

Praktikan diminta untuk memahami kembali bagaimana menjalankan perintah TTS melalui terminal, kemudian mengadaptasikannya ke dalam kode Python yang telah disediakan.

Hint: Perhatikan konfigurasi seperti path model, konfigurasi file, serta speaker yang digunakan.

Pastikan semua variabel telah ditulis dengan tepat agar tidak terjadi error saat proses konversi teks ke suara dilakukan.

Endpoint Utama

File `main.py` berfungsi sebagai titik masuk utama (entry point) dari backend aplikasi. File ini menangani seluruh proses alur utama dari sistem, dimulai dari menerima file audio dari pengguna, kemudian mengonversinya menjadi teks (STT), mengirim teks tersebut ke Gemini (LLM) untuk mendapatkan respons, dan terakhir mengubah respons tersebut menjadi file audio (TTS) yang dikirimkan kembali ke frontend. Semua komunikasi dilakukan menggunakan framework `FastAPI` yang ringan dan cepat.

Endpoint utama pada aplikasi ini adalah `/voice-chat` dengan metode `POST`. Endpoint ini dirancang untuk menerima file audio yang dikirimkan dari frontend, memprosesnya melalui serangkaian fungsi backend, dan mengembalikan file audio hasil respons ke pengguna. Endpoint ini juga secara otomatis akan mengembalikan file audio hasil konversi dengan menggunakan kelas `FileResponse` dari `fastapi.responses`, sehingga pengguna dapat langsung memutar atau mengunduh hasil suara dari chatbot.

Pada file `main.py`, praktikan diminta untuk melengkapi logika utama pada endpoint `/voice-chat`. Beberapa bagian kode seperti pemanggilan fungsi dari `stt.py`, `llm.py`, dan `tts.py` perlu diisi dengan benar agar alur kerja backend dapat berjalan sesuai yang diharapkan.

Menjalankan Backend

Setelah pengisian kode pada file `main.py` selesai dilakukan, jalankan backend aplikasi menggunakan perintah berikut melalui terminal:

```
$ uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

Pastikan bahwa Anda sudah berada pada direktori root proyek dan environment virtual sudah dalam kondisi aktif sebelum menjalankan perintah di atas. Jika backend berhasil dijalankan, Anda akan melihat log server yang menunjukkan bahwa aplikasi berjalan di alamat `http://127.0.0.1:8000`.

Untuk menguji apakah sistem telah berjalan dengan benar, Anda bisa mengaktifkan langsung menggunakan frontend yang telah tersedia.

Menjalankan Frontend

Frontend dari aplikasi ini dibangun menggunakan pustaka `Gradio`, yaitu sebuah framework Python yang memungkinkan pembuatan antarmuka pengguna (UI) interaktif secara cepat dan sederhana, khususnya untuk aplikasi berbasis machine learning. `Gradio` memiliki konsep yang serupa dengan `Streamlit`, yaitu menyederhanakan proses pembuatan frontend sehingga pengguna hanya perlu fokus pada logika program tanpa harus menulis kode HTML atau JavaScript secara langsung. `Gradio` sangat cocok digunakan saat ingin menguji dan memvisualisasikan model secara langsung dari browser.

Untuk menjalankan frontend, pastikan Anda telah berada di dalam direktori `gradio_app/`, kemudian jalankan perintah berikut:

```
$ python app.py
```

Jika berhasil dijalankan, `Gradio` akan memberikan alamat lokal (biasanya `http://127.0.0.1:7860`) yang dapat Anda buka melalui browser untuk mengakses antarmuka aplikasi. Pastikan backend sudah berjalan sebelum membuka frontend agar seluruh fitur dapat digunakan secara penuh. Jika aplikasi sudah berjalan, silahkan masukkan pertanyaan melalui suara dan tunggu beberapa saat untuk mendapatkan jawaban dalam bentuk suara juga.